Alunos: Laerte de Lara e Arthur Macedo

Atividade Aula 3 – Padrões de Desenvolvimento de Software

Parte 1 – Identificação das Violações

- Violação do ISP

Trecho violado: A interface Funcionario agrupa 5 métodos num contrato único.

Por que é violação: Classes são forçadas a implementar métodos que não fazem sentido para elas. Gerente não escreve código, Desenvolvedor não gerencia equipe, Estagiario não recebe salário nem gerencia — mas todos são obrigados a declarar esses métodos.

Impacto: O sistema fica frágil e enganoso — um método existe na interface mas lança erro em tempo de execução, em vez de não existir no contrato.

- Violação do LSP

Trecho violado:

- Gerente.escreverCodigo() → throw new Error("Gerente não escreve código")
- Desenvolvedor.gerenciarEquipe() → throw new Error("Dev não gerencia equipe")
- Estagiario.receberSalario() → throw new Error("Estagiário recebe bolsa, não salário")
- Estagiario.gerenciarEquipe() → throw new Error("Estagiário não gerencia")

Por que é violação: Uma subclasse que lança Error para um método herdado viola o contrato da superclasse/interface. Não é possível substituir Funcionario por Estagiario com segurança — o programa quebra em runtime.

Impacto: Qualquer código que itere sobre uma lista de Funcionario[] e chame receberSalario() vai lançar exceção inesperada se houver um Estagiario na lista.

- Violação do DIP

Trecho violado: Não existe uma classe SistemaRH. Se houvesse, ela dependeria diretamente das classes concretas (Gerente, Desenvolvedor, Estagiario).

Por que é violação: Módulos de alto nível (ex: SistemaRH) não devem depender de implementações concretas, mas de abstrações (interfaces).

Impacto: Adicionar um novo tipo de funcionário exigiria modificar a classe SistemaRH — violando também o Open/Closed Principle.

Parte 2 e 3 – Código Refatorado (com Freelancer)

typescript

```
// PARTE 2 – ISP: Interfaces segregadas por responsabilidade

interface Trabalhavel {
  trabalhar(): void;
}

interface RegistraPonto {
  registrarPonto(): void;
}

interface Assalariado {
  receberSalario(): void;
}

interface Gerenciavel {
  gerenciarEquipe(): void;
}

interface Programavel {
```

```
  escreverCodigo(): void;
}

interface Remuneravel {
  receberPagamento(): void; // abstração genérica para salário OU pagamento por projeto
}

// PARTE 2 – LSP: Nenhuma classe lança Error para método implementado

class Gerente implements Trabalhavel, RegistraPonto, Assalariado, Gerenciavel, Remuneravel {
  trabalhar(): void {
    console.log("Gerente trabalhando");
  }
  registrarPonto(): void {
    console.log("Ponto registrado");
  }
  receberSalario(): void {
    console.log("Salário recebido");
  }
  gerenciarEquipe(): void {
    console.log("Gerenciando equipe");
  }
  receberPagamento(): void {
    this.receberSalario();
  }
}
```

```
class Desenvolvedor implements Trabalhavel, RegistraPonto, Assalariado, Programavel, Remuneravel {
  trabalhar(): void {
    console.log("Desenvolvedor trabalhando");
  }
  registrarPonto(): void {
    console.log("Ponto registrado");
  }
  receberSalario(): void {
    console.log("Salário recebido");
  }
  escreverCodigo(): void {
    console.log("Escrevendo código");
  }
  receberPagamento(): void {
    this.receberSalario();
  }
}

class Estagiario implements Trabalhavel, RegistraPonto, Programavel, Remuneravel {
  trabalhar(): void {
    console.log("Estagiário trabalhando");
  }
  registrarPonto(): void {
    console.log("Ponto registrado");
  }
  receberBolsa(): void {
    console.log("Bolsa recebida");
  }
```

```
  escreverCodigo(): void {
    console.log("Estagiário escrevendo código");
  }
  receberPagamento(): void {
    this.receberBolsa();
  }
}

// PARTE 3 – Freelancer: adicionado SEM modificar nenhuma classe existente



class Freelancer implements Trabalhavel, Programavel, Remuneravel {
  trabalhar(): void {
    console.log("Freelancer trabalhando");
  }
  escreverCodigo(): void {
    console.log("Freelancer escrevendo código");
  }
  receberPorProjeto(): void {
    console.log("Pagamento por projeto recebido");
  }
  receberPagamento(): void {
    this.receberPorProjeto();
  }
}
```

```
// PARTE 2 – DIP: SistemaRH depende de abstrações, não de concretos

class SistemaRH {
  private funcionarios: Remuneravel[];
  private trabalhadores: Trabalhavel[];

  constructor(trabalhadores: Trabalhavel[], funcionarios: Remuneravel[]) {
    this.trabalhadores = trabalhadores;
    this.funcionarios = funcionarios;
  }

  executarJornada(): void {
    console.log("\n--- Iniciando jornada ---");
    this.trabalhadores.forEach(t => t.trabalhar());
  }

  processarPagamentos(): void {
    console.log("\n--- Processando pagamentos ---");
    this.funcionarios.forEach(f => f.receberPagamento());
  }
}
// DEMONSTRAÇÃO – O Freelancer entra sem nenhuma modificação anterior

const gerente     = new Gerente();
const dev         = new Desenvolvedor();
const estagiario  = new Estagiario();
const freelancer  = new Freelancer(); // ← novo tipo, zero mudanças nas classes existentes
```

```
const sistema = new SistemaRH(
  [gerente, dev, estagiario, freelancer],   // Trabalhavel[]
  [gerente, dev, estagiario, freelancer]    // Remuneravel[]
);

sistema.executarJornada();
sistema.processarPagamentos();
```